

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 9 Appendix: The Landscape (Dated), Setup Details, References

● Zhigang Feng

2026-09-13 15:52:29

What This Appendix Holds

Everything here is dated. The main decks point to this appendix instead of repeating product facts — so this is the one deck to re-check before teaching.

1. **The landscape, dated** — the interoperability stack and neutral governance, frameworks, reasoning models, scheduled runtimes, computer-use agents
2. **Setup details** — installing each supported harness, the lab's setup check, keys and a minimal instructions file, dated comparisons
3. **Resources** — documentation, economists' public skill repositories, the security and reproducibility references, the Context Dilemma readings

Reading convention: [solid] = primary or citable; [hype] = vendor self-report. Harness facts come from CLI_FACTS_2026-09.md in the lecture's source folder, which records how each was checked.

Checked on 12 September 2026. The field moves monthly: re-run the checks before teaching.

The Landscape (Dated)

Compete on models, cooperate on plumbing — as of September 2026

Three Layers of Agent Interoperability

The 2025–26 story is standard-setting: vendors compete on models but increasingly cooperate on the wiring between agents and tools.

Layer	Protocol	What it standardizes
Agent ↔ tool	MCP (Model Context Protocol)	how an agent discovers and calls external tools/data sources
Agent ↔ agent	A2A (Agent-to-Agent)	how independent agents delegate tasks to one another
Discovery / identity	AGNTCY / ACP	agent directories, capability cards, identity and trust

- MCP was introduced by Anthropic (Nov 2024); it is now *multi-vendor*, not proprietary.
- **OpenAI announced MCP support in March 2025**, starting with its Agents SDK; Google and Microsoft tooling support it too. **[solid]**
- Governance moved to a neutral home: the **Linux Foundation Agentic AI Foundation (AAIF)**, formed Dec 9, 2025, now hosts MCP and related projects. **[solid]**

Why Neutral Governance Is a Network-Effects Story

For economists, the interoperability stack is a textbook standard-setting / network-effects case.

- A shared protocol lowers switching costs: a tool wrapped once in MCP works across Claude, ChatGPT, Gemini, and open agents
- Network effects make the standard more valuable as adoption grows — which is why a single-vendor protocol faces adoption resistance
- Moving MCP under the **Linux Foundation AAIF** (Dec 9, 2025) is the classic move: hand the “plumbing” to a neutral body so rivals will build on it
- Competition then concentrates where rents remain: model quality, latency, and price — not the connectors

Reproducibility note. Protocols version. Pin the *spec version* you used (e.g. MCP spec date, A2A version) alongside the model version and date — a tool call that worked under one spec revision may behave differently under the next.

Frameworks Reached 1.0 (2025–26)

The orchestration layer matured from research demos to versioned, supported frameworks.

Framework	Milestone	Vendor
LangGraph 1.0	Oct 2025	LangChain
OpenAI Agents SDK (+ AgentKit)	SDK Mar 2025	OpenAI
Google Agent Dev. Kit (ADK)	v1.0 May 2025	Google
Microsoft Agent Framework	1.0 Apr 2026	Microsoft
CrewAI	1.0	CrewAI
Anthropic Claude Agent SDK	Sep 2025	Anthropic

Naming note. What was the *Claude Code SDK* was renamed the **Claude Agent SDK** on Sep 29, 2025, reflecting generalization beyond coding. **[solid]**

Durable lesson: the patterns outlive the framework. Learn orchestrator–worker once; map it onto whichever SDK your institution adopts.

Reasoning Models and Think-Budget Control

Frontier models now spend variable inference-time compute, and increasingly call tools inside the reasoning loop.

- **Tool use in the loop.** OpenAI o3 / o4-mini (Apr 2025) interleave tool calls with chain-of-thought rather than reasoning first and acting after.
- **Think-budget routers.** GPT-5 (Aug 2025) → GPT-5.5 (Apr 2026) route between fast and deep modes per query.
- **Toggleable extended thinking + configurable token budget.** Claude Opus 4.5 (Nov 2025) lets you turn extended thinking on/off and cap the thinking-token budget; it was the first model past **80% on SWE-bench Verified**, and Opus pricing was cut to \$5/\$25 per Mtok (input/output).
- **Parallel deep reasoning.** Gemini “Deep Think” explores multiple reasoning paths in parallel.

Benchmark caveat (carry this everywhere). The 80% SWE-bench Verified figure is real but the metric is contaminated/saturating: OpenAI *stopped reporting* SWE-bench Verified in Feb 2026. Pair any Verified headline with SWE-bench Pro (~55–59% at the frontier, held-out). **[solid]**

Coding Agents Became Scheduled, Autonomous Runtimes

In 2025–26, coding agents shifted from interactive assistants to background runtimes with persistent state.

- **OpenAI Codex “Goal Mode”** became the default (May 2026), with persistent cross-session state — the agent resumes a goal rather than starting fresh each chat.
- **Anthropic “Code with Claude” (May 6, 2026)** reframed Claude Code as an *orchestrator*:
 - **Auto Mode** with prompt-injection / destructive-action classifiers + human approval gates
 - **Routines** (launched 14 Apr 2026): runs on a schedule, an API call, or a GitHub webhook
 - **Worktrees** for parallel branches
 - sandboxed **Managed Agents** with scoped credentials

Lesson for research infra. Before an agent runs unattended on real data or repos, configure sandboxing, credential scoping, and approval gates first. “Autonomous” is a permissions decision, not a model capability.

Computer-Use Agents: Strong Benchmarks, Volatile Products

Browser/computer-use agents got good on paper but the standalone products churned.

- Computer-use / browser agents reached **~78–84% on OSWorld-Verified** (human baseline ~72%).
- **But standalone products are volatile:** Google *killed* Project Mariner (May 2026); OpenAI folded Operator into ChatGPT Agent / Atlas.

Flag the benchmark numbers. OSWorld scores are **scaffold-dependent and largely self-reported**: the same model scores very differently under different harnesses, and “above human baseline” does not mean reliable on *your* unseen workflow. Treat as directional. **[solid claim that products churned; benchmark figures = vendor/scaffold-dependent]**

Lesson: do not build durable research infrastructure on a single experimental product. Build on the protocol layer (MCP) and your own files; treat any one browser agent as replaceable.

Synthesis: The Landscape Reinforces the Workflow Spine

Every module above maps back onto the terminal-agent workflow of T1–T5d and onto the course's measurement / identification theme.

Landscape module	Connection to the spine
Interoperability (MCP/A2A)	build on the neutral protocol layer, not one product; pin spec versions
Orchestration	T5b's sub-agent review <i>is</i> orchestrator–worker; pay for multi-agent only when value > token cost
Reasoning & time horizons	"runs for hours" → a measured, reliability-thresholded number
Coding / computer-use	autonomy is a sandboxing + approval-gate decision, set before the run
Benchmarks	a headline score is not a measurement until you know its validity (Verified vs. Pro)
Safety & reproducibility	untrusted-input discipline + logged chain-of-custody (dates, versions, seeds)

The plumbing changes monthly; the research discipline does not.

Compete on questions and judgment; cooperate on standards; measure before you trust.

Setup Details

Dated install notes, keys, and templates

Appendix A: Installing a Terminal Agent (Dated)

Install one — any of the three the course supports. The shape is the same: install, sign in once, start from the project folder.

```
# Claude Code (native installer; npm also works)
curl -fsSL https://claude.ai/install.sh | bash
claude # then /login

# Codex CLI (needs Node.js)
npm install -g @openai/codex
codex login

# Gemini CLI (needs Node.js)
npm install -g @google/gemini-cli
gemini # sign in on first start; trust the project folder
```

- **Verify:** `claude --version`, `codex --version`, or `gemini --version` — then `cd` into the project and start the agent there
- **Mac + OneDrive:** launch from the project folder itself; a `/Volumes/...` path can trigger permission errors

Versions on the course machine, 12 Sep 2026: Claude Code 2.1.270, Codex CLI 0.154.0, Gemini CLI 0.47.0. Plans, tiers, and commands change every few months — check the official docs.

Appendix A2: Setup for the Lab

Lightweight setup checklist for a terminal-agent workflow

- Install one terminal agent, then run the lab's setup check: `python3 scripts/setup_check.py --live`
- Create a project folder with a project-specific system prompt (e.g., `CLAUDE.md`, `AGENTS.md`, `GEMINI.md`), `scripts/`, and `output/`
- Make sure Python or R runs from the terminal
- Keep the first session small: one file, one check, one output

Dated note: exact setup steps, plans, and rate limits change quickly. Recheck the official docs before the lab.

Appendix B: Keybindings and Minimal Templates

Useful commands

Compact session	/compact
Clear session	/clear
Interrupt	Esc
Slash menu	/

Minimal project-system-prompt

```
# CLAUDE.md / AGENTS.md / GEMINI.md
Project: ...
Goal: ...
Language: ...
Checks: ...
```

Appendix C: Dated Comparisons

The numbers the main decks deliberately leave out, with the date each was last checked. Replace them; do not quote them old.

What	As last recorded	Status
Terminal agents in the course	Claude Code 2.1.270, Codex CLI 0.154.0, Gemini CLI 0.47.0 (Aider also exists)	checked 12 Sep 2026
METR 50%-reliability time horizons (T1)	Time Horizon 1.1, Jan 2026: Opus 4.5 $\approx$ 320 min, GPT-5 $\approx$ 214 min	stale : predates current models — refresh
T1 timeline: tool calls	OpenAI function calling, June 2023	[solid]
T1 timeline: shared plumbing	MCP introduced by Anthropic, Nov 2024; OpenAI support from March 2025; neutral foundation (AAIF), Dec 2025	checked 12 Sep 2026
T1 timeline: terminal agents	Claude Code, Codex CLI, Gemini CLI all released in 2025	re-verify months
T1 timeline: unattended runs	Claude Code routines, 14 Apr 2026; Codex Goal Mode generally available, May 2026	checked 12 Sep 2026 (Codex: press reports)
T3b sidebar: ten thousand agents	OpenAI, Navier–Stokes: $\sim$ 10,000 agents from 1 Sep 2026; a proof in 88 h, then 17 h in Lean (vendor report)	[hype]; re-verify at the gate

Teaching rule: dated comparisons live here, not in the conceptual core of the lecture.

Resources

Docs, repositories, and references



Resources

Where to read further — each chosen because it helps you check an agent's work, not just use one.

Using agents as an economist

- Official documentation for Claude Code, Codex CLI, and Gemini CLI
- Pedro Sant'Anna, github.com/pedrohcg/claude-code-my-workflow; Chris Blattman, github.com/chrisblattman/claudeblattman
- Aniket Panjwani, "AI Agents for Economics Research" (CEPR/VoxDev webinar, 2026)

Checking agents

- METR, measured task horizons (metr.org)
- OWASP, *Top 10 for Agentic Applications* (Dec 2025): the risks of agents that plan, remember, and act
- Siegel et al. (2024), *CORE-Bench*: agents reproducing published results from code and data, social science included
- OpenAI (Feb 2026) on retiring SWE-bench Verified; Berkeley RDI (Apr 2026) on breaking agent benchmarks
- The AEA's journal policies on AI use and on data and code

And this course's own: the Lecture 9 lab ([labs/Lec09_Agent_Lab/](#)), its recorded transcripts, and *CLI_FACTS_2026-09.md*. Checked 12 Sep 2026.

Appendix D: Context Dilemma — References

Context as implicit prior

- Xie, Raghunathan, Liang, Ma (2022). An Explanation of In-context Learning as Implicit Bayesian Inference. ICLR. arXiv:2111.02080.

Sycophancy and RLHF limits

- Sharma et al. (2023). Towards Understanding Sycophancy in Language Models. arXiv:2310.13548.
- Perez et al. (2023). Discovering Language Model Behaviors with Model-Written Evaluations. ACL Findings. arXiv:2212.09251.
- Casper et al. (2023). Open Problems and Fundamental Limitations of RLHF. TMLR. arXiv:2307.15217.
- Wolf et al. (2023). Fundamental Limitations of Alignment in Large Language Models. arXiv:2304.11082.

Self-correction and adversarial review

- Huang et al. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. ICLR. arXiv:2310.01798.
- Valmeekam et al. (2023). Can Large Language Models Really Improve by Self-critiquing Their Own Plans? NeurIPS. arXiv:2310.08118.
- Du et al. (2023). Improving Factuality and Reasoning through Multiagent Debate. arXiv:2305.14325.
- Irving, Christiano, Amodei (2018). AI Safety via Debate. arXiv:1805.00899.

Monoculture and the economist workflow

- Kleinberg & Raghavan (2021). Algorithmic Monoculture and Social Welfare. PNAS 118(22).
- Bommasani et al. (2022). Picking on the Same Person: Does Algorithmic Monoculture Lead to Outcome Homogenization? NeurIPS. arXiv:2211.13972.
- Korinek (2023). Language Models and Cognitive Automation for Economic Research. NBER WP 30957; JEL.